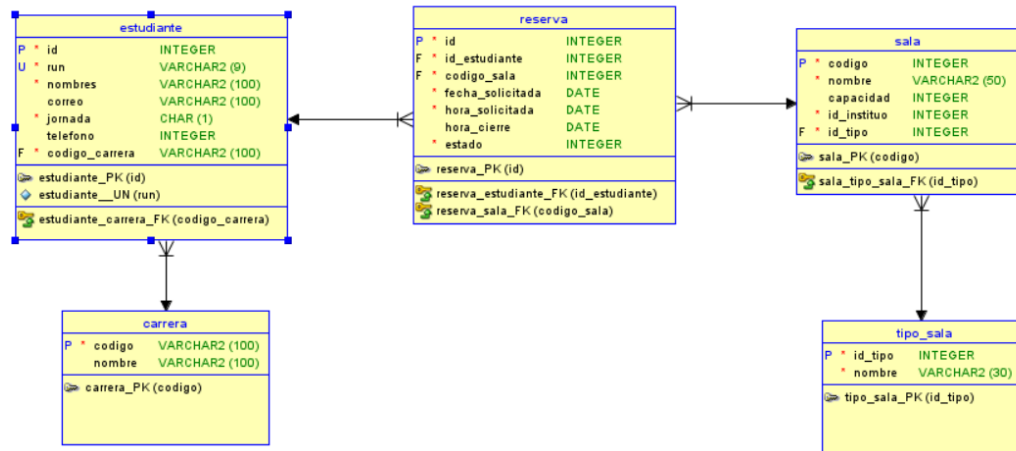


Guía práctica

Proyecto reserva de salas biblioteca

Análisis a los test

En esta guía, detallaremos paso a paso el desarrollo de un proyecto en Spring llamado "Salas Bibliotecas".



Plan de Prueba de Testing con JUnit 5 para el Proyecto de Reserva de Salas

Introducción

El plan de prueba pretende verificar la funcionalidad de los servicios del sistema de reserva de salas. Se utilizará **JUnit 5** para las pruebas unitarias y **Mockito** para la simulación de dependencias.

Estructura de la Base de Datos

La estructura de la base de datos incluye las siguientes tablas:

- Carrera
- Estudiante
- Reserva
- Sala
- Tipo de Sala

Configuración de Pruebas

Dependencias Maven: agregar las dependencias necesarias en el archivo pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-engine</artifactId>
  <scope>test</scope>
</dependency>
```

Problemas al ejecutar un test ✕

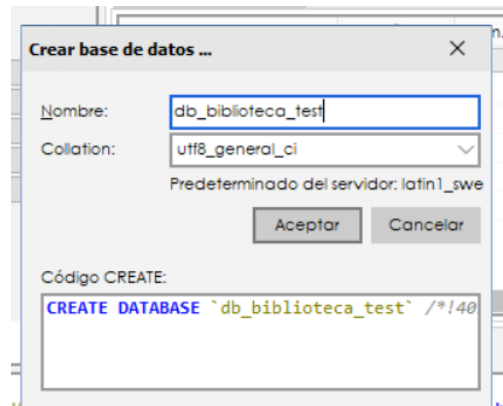
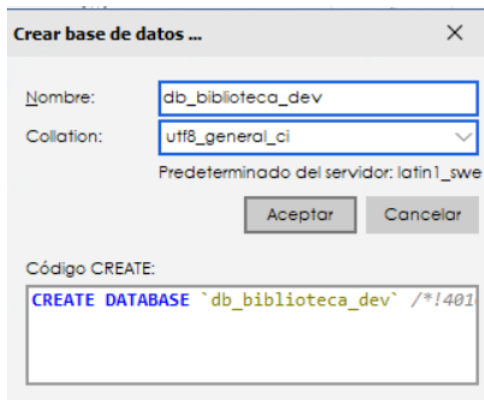
Cuando realizamos una prueba de testing, estaremos ejecutando en la misma base de datos. Para esto se crea una base de datos de test y otra de desarrollo.

Para separar la base de datos local de la de pruebas en un proyecto Spring Boot, se pueden usar diferentes configuraciones en el archivo application.properties o application.yml para cada perfil de entorno (e.g., dev para desarrollo y test para pruebas). Esto asegura que se utilicen bases de datos diferentes para el desarrollo y para ejecutar las pruebas unitarias.

Configuración de la Base de Datos

Archivo de Configuración para Desarrollo (application-dev.properties):

- Crear dos archivos nuevos application-dev.properties y application-test.properties
- Crear dos bases de datos



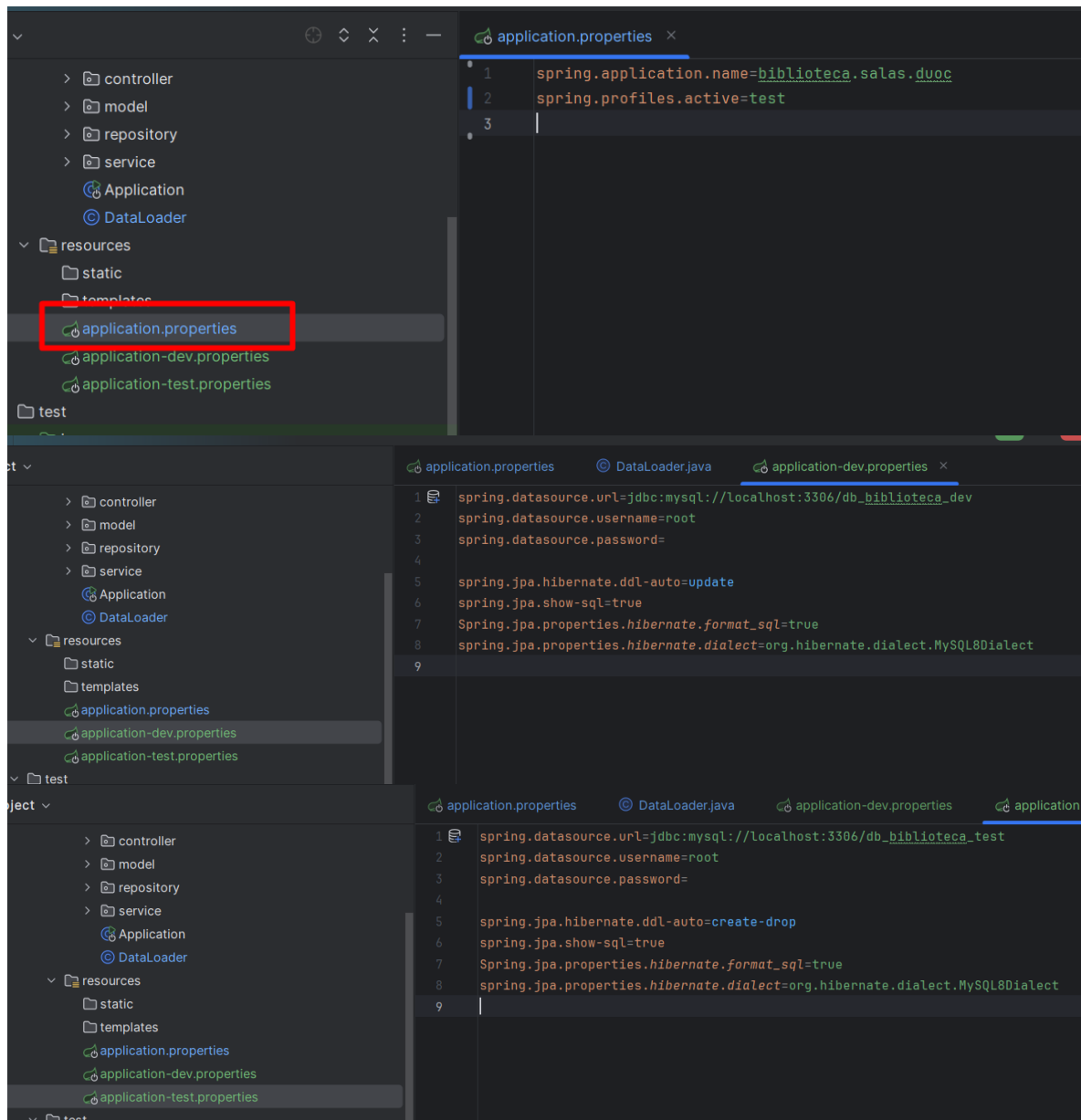
src/main/resources/application-dev.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/mi_base_datos_dev
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

src/main/resources/application-test.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/mi_base_datos_test
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

Entonces nos queda así.



The image shows three screenshots of an IDE (IntelliJ IDEA) illustrating the configuration of Spring properties files for different environments.

Top Screenshot: The file explorer on the left shows the project structure. The file `application.properties` is highlighted with a red box. The editor on the right shows the content of `application.properties`:

```
1 spring.application.name=biblioteca.salas.duoc
2 spring.profiles.active=test
3
```

Middle Screenshot: The file explorer on the left shows the project structure. The file `application-dev.properties` is highlighted. The editor on the right shows the content of `application-dev.properties`:

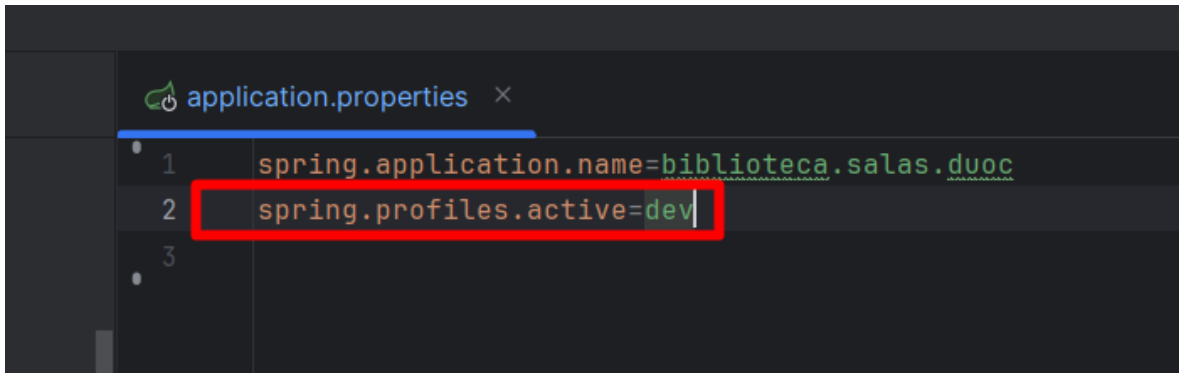
```
1 spring.datasource.url=jdbc:mysql://localhost:3306/db_biblioteca_dev
2 spring.datasource.username=root
3 spring.datasource.password=
4
5 spring.jpa.hibernate.ddl-auto=update
6 spring.jpa.show-sql=true
7 Spring.jpa.properties.hibernate.format_sql=true
8 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
9
```

Bottom Screenshot: The file explorer on the left shows the project structure. The file `application-test.properties` is highlighted. The editor on the right shows the content of `application-test.properties`:

```
1 spring.datasource.url=jdbc:mysql://localhost:3306/db_biblioteca_test
2 spring.datasource.username=root
3 spring.datasource.password=
4
5 spring.jpa.hibernate.ddl-auto=create-drop
6 spring.jpa.show-sql=true
7 Spring.jpa.properties.hibernate.format_sql=true
8 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
9
```

Para cambiar de test a desarrollo es solo ir cambiando, si queremos ejecutar test cambiar a test.

spring.profiles.active=test o spring.profiles.active=dev



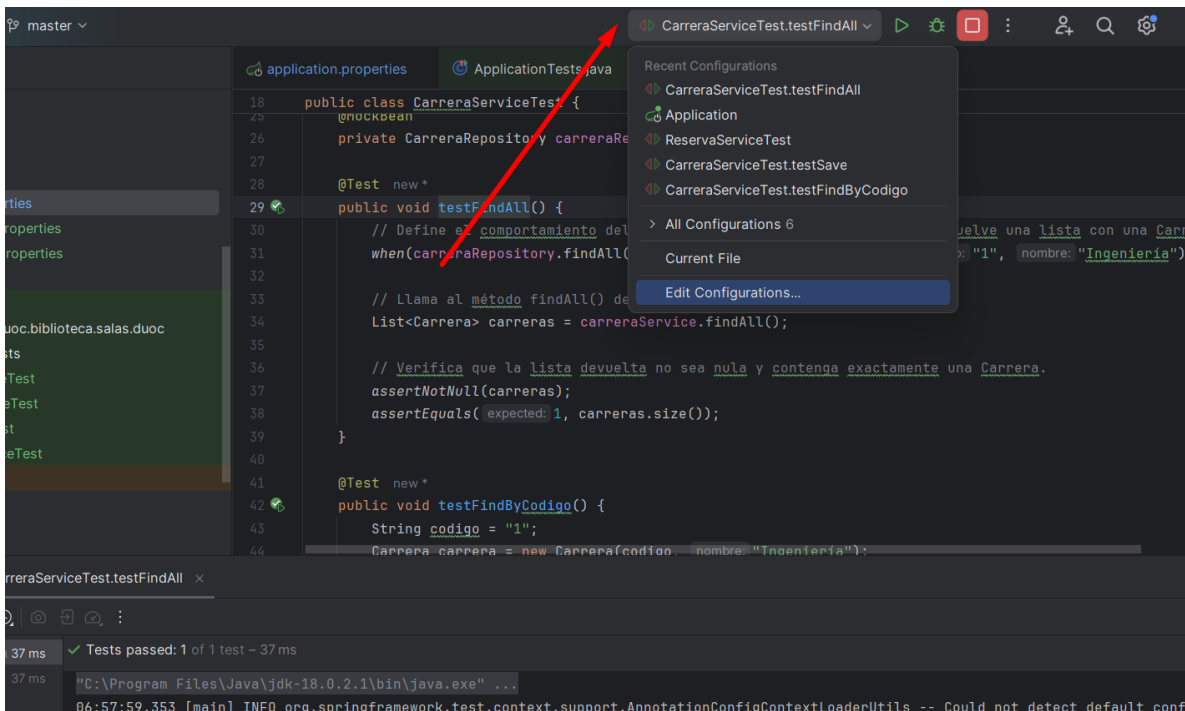
```

1  spring.application.name=biblioteca.salas.duoc
2  spring.profiles.active=dev
3

```

Ejecutar los test

Se puede ejecutar las pruebas de manera individual



```

18  public class CarreraServiceTest {
19      @MockBean
20      private CarreraRepository carreraRepository;
21
22      @Test
23      public void testFindAll() {
24          // Define el comportamiento del mock
25          when(carreraRepository.findAll()).thenReturn(listOf(
26              // Llama al método findAll() de CarreraService
27              List<Carrera> carreras = carreraService.findAll();
28
29              // Verifica que la lista devuelta no sea nula y contenga exactamente una Carrera.
30              assertNotNull(carreras);
31              assertEquals(expected: 1, carreras.size());
32          });
33
34      @Test
35      public void testFindByCodigo() {
36          String codigo = "1";
37          Carrera carrera = new Carrera(codigo, nombre: "Ingenieria");
38      }
39  }

```

Recent Configurations

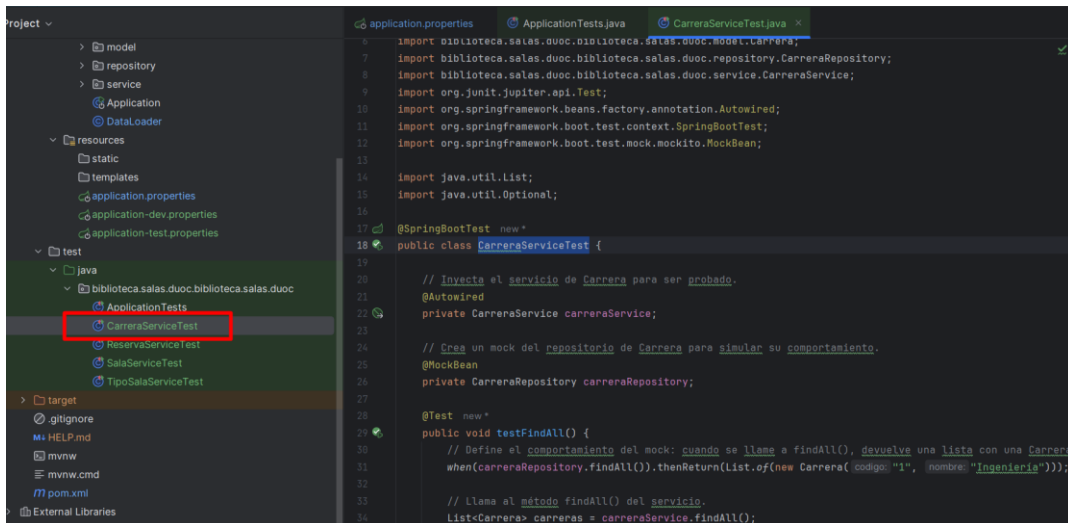
- CarreraServiceTest.testFindAll
- Application
- ReservaServiceTest
- CarreraServiceTest.testSave
- CarreraServiceTest.testFindByCodigo
- > All Configurations 6
- Current File
- Edit Configurations...

Tests passed: 1 of 1 test - 37 ms

Análisis de testing nivel servicio:

Descomprime el zip y analiza junto a tu equipo, los diferentes pruebas realizadas y completa las preguntas, de la guía.

TESTING - Analizar Services Test - biblioteca.salas.duoc.zip



The screenshot shows an IDE with the project structure on the left and the code of `CarreraServiceTest.java` on the right. The project structure includes:

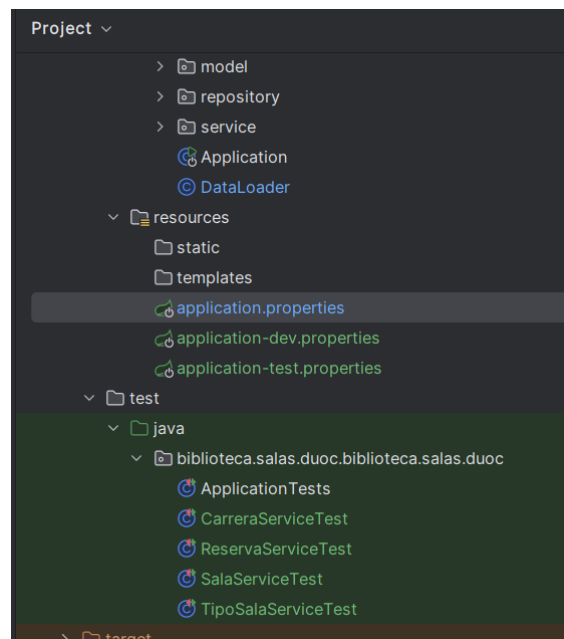
- model
- repository
- service
 - Application
 - DataLoader
- resources
 - static
 - templates
 - application.properties
 - application-dev.properties
 - application-test.properties
- test
 - java
 - biblioteca.salas.duoc.biblioteca.salas.duoc
 - ApplicationTests
 - CarreraServiceTest** (highlighted with a red box)
 - ReservaServiceTest
 - SalaServiceTest
 - TipoSalaServiceTest

The code in `CarreraServiceTest.java` is as follows:

```

1  import biblioteca.salas.duoc.biblioteca.salas.duoc.repository.CarreraRepository;
2  import biblioteca.salas.duoc.biblioteca.salas.duoc.service.CarreraService;
3  import org.junit.jupiter.api.Test;
4  import org.springframework.beans.factory.annotation.Autowired;
5  import org.springframework.boot.test.context.SpringBootTest;
6  import org.springframework.boot.test.mock.mockito.MockBean;
7
8  import java.util.List;
9  import java.util.Optional;
10
11 @SpringBootTest
12 public class CarreraServiceTest {
13
14     // Inventa el servicio de Carrera para ser probado.
15     @Autowired
16     private CarreraService carreraService;
17
18     // Crea un mock del repositorio de Carrera para simular su comportamiento.
19     @MockBean
20     private CarreraRepository carreraRepository;
21
22     @Test
23     public void testFindAll() {
24         // Define el comportamiento del mock: cuando se llame a findAll(), devuelve una lista con una Carrera
25         when(carreraRepository.findAll()).thenReturn(List.of(new Carrera(codigo "1", nombre "Ingeniería")));
26
27         // Llama al método findAll() del servicio.
28         List<Carrera> carreras = carreraService.findAll();
29     }
30 }

```



```
@Autowired
private CarreraService carreraService;

// Crea un mock del repositorio de Carrera para simular su comportamiento.
@MockBean
private CarreraRepository carreraRepository;

@Test
public void testFindAll() {
    // Define el comportamiento del mock: cuando se llame a findAll(), devuelve una lista con una Carrera.
    when(carreraRepository.findAll()).thenReturn(List.of(new Carrera( "1", "Ingeniería")));

    // Llama al método findAll() del servicio.
    List<Carrera> carreras = carreraService.findAll();

    // Verifica que la lista devuelta no sea nula y contenga exactamente una Carrera.
    assertNotNull(carreras);
    assertEquals( expected: 1, carreras.size());
}
```

¿Qué hace `testFindAll()`?

El propósito de este método es verificar que el método `findAll` del servicio `CarreraService` funcione correctamente. Específicamente, se asegura de que el método `findAll` devuelva una lista de todas las carreras almacenadas en el repositorio.

Pasos Detallados

Configuración del Mock (when...thenReturn):

- `when(carreraRepository.findAll()).thenReturn(List.of(new Carrera("1", "Ingeniería")));`
- En este paso, se configura el comportamiento simulado (mock) del `carreraRepository`.
- Específicamente, se indica que cuando el método `findAll()` del `carreraRepository` sea llamado, debe devolver una lista que contiene un solo objeto `Carrera` con código "1" y nombre "Ingeniería".

Llamada al Método del Servicio:

- `List<Carrera> carreras = carreraService.findAll();`
- Aquí se llama al método `findAll()` del `carreraService`.
- El servicio, a su vez, llama al `findAll()` del repositorio.
- Debido a la configuración del mock en el paso anterior, el repositorio devolverá la lista simulada de carreras.

Verificación de la Lista Devuelta:

- `assertNotNull(carreras);`
- Este paso verifica que la lista de carreras devuelta no sea nula. Esto asegura que el método `findAll` del servicio realmente devuelve una lista.
- `assertEquals(1, carreras.size());`

Este paso verifica que la lista contenga exactamente un elemento. Esto asegura que la lista de carreras devuelta por el servicio contiene el número esperado de elementos.

¿Analiza los demás testing y comenta con tu compañero su implementación a los servicios?

¿Porque se implementan testing a nivel de servicio, se pueden implementar a nivel de controller?

Análisis de testing nivel de controller:



TESTING - CONTROLLER - bibliteca.salas.duoc.zip

¿Qué se espera realizar en este testing?

Analiza la prueba estudiante Controller ¿Qué se espera realizar en el **testCreateEstudiante**?

Ejecuta todos los test. ¿Existe alguna falla? Corrige la falla y comenta ¿qué era?

?? **BUEN TRABAJO** ??